

Facultad de Ingeniería, Universidad de Buenos Aires

Sistemas Distribuidos

TP diseño

Grupo 15

Integrante	**Padrón**
Basso, Catalina	108564
Favotti, Franco	109388
Fernandez Delgado, Victoria	110545

11 de octubre de 2026

Índice

1. Introducción **2**
1.1. Descripción del sistema 2
1.2. Consultas a resolver 2

2. Vista de Arquitectura **2**

3. Escenarios **3**
3.1. Casos de Uso 3

4. Vista Lógica **4**
4.1. DAG 4

5. Vista de Procesos **4**
5.1. Diagrama de actividades 4
5.2. Diagrama de secuencia 5

6. Vista de Desarrollo **6**
6.1. Diagrama de Paquetes 6

7. Vista Física **7**
7.1. Diagrama de Robustez 7
7.2. Diagrama de Despliegue 8

8. Tareas y división entre integrantes **11**

A. Diagrama de secuencia completo **12**

1 Introducción

En este trabajo práctico, se diseña un sistema para analizar un conjunto de artículos de Wikipedia en Ingles y en Francés. Se utiliza el modelo de vista 4+1 para presentarlo.

1.1 Descripción del sistema

El sistema recibe un conjunto de artículos desde uno o mas clientes, los procesa de forma distribuida mediante un pipeline de etapas conectadas por cola y devuelve a cada cliente los resultados de las cinco consultas.

1.2 Consultas a resolver

El análisis se realiza mediante consultas, que se resuelven de distintas maneras.

1. Url y fecha de modificación para artículos escritos en inglés hasta 2020 inclusive.
2. Nombre, url e imágen (bytes) de artículos escritos hasta 2020 inclusive que incluyan “biography” o “biographie” entre sus secciones.
3. Listado de palabras únicas compartidas entre cualquier artículo escrito en inglés y cualquier artículo escrito en francés.
4. Pares (palabra, cantidad) del top 20 palabras más utilizadas entre todos los abstracts. Dentro de un mismo abstract las palabras se cuentan una sola vez y las palabras triviales (stopwords) se excluyen del top.
5. Url y cantidad de palabras de abstract para los artículos escritos a partir del 2021 cuya cantidad de palabras en el abstract sea superior al promedio de palabras de abstract de todos los artículos escritos hasta el 2020 inclusive. Se aplica el mismo criterio para contabilizar palabras que la query anterior.

2 Vista de Arquitectura

El cliente se conecta mediante TCP a un Gateway, que recibe los batches de artículos y, a través de su `MessageHandler`, los publica en las colas de entrada de cada rama: `q1_in` (Q1), `q2_in` (Q2) y `norm_in` (procesamiento de texto para Q3, Q4 y Q5). En cada etapa de procesamiento hay réplicas que consumen en paralelo de colas del middleware del sistema (RabbitMQ); solo las etapas de agregación global (`TopMerger`, `AverageAggregator` y `Joiner`) tienen una única instancia. Los resultados de las cinco consultas llegan a una única cola hacia el Joiner, que los persiste y los envía al cliente a través del Gateway.

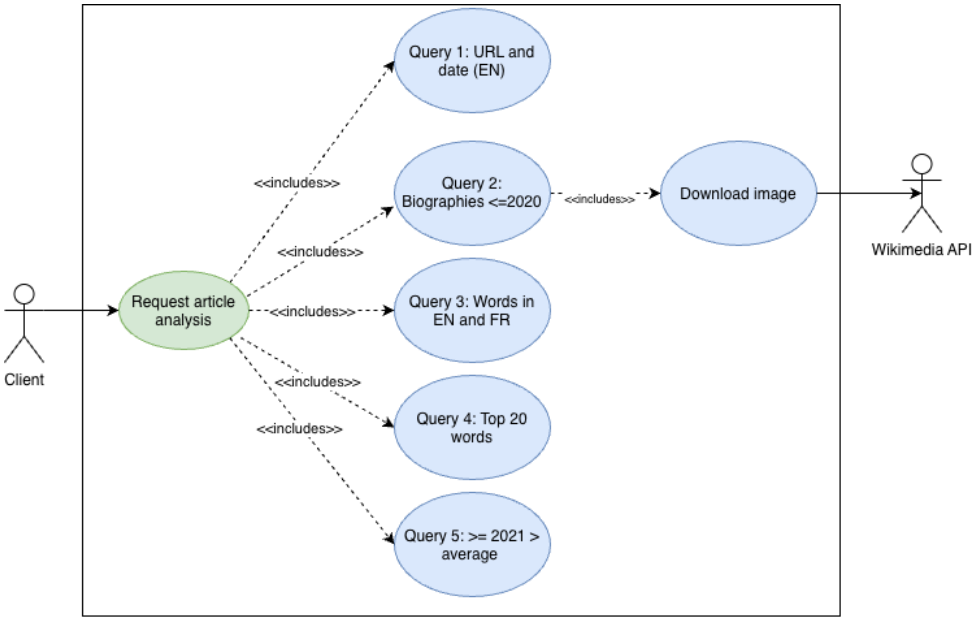
Componente	Responsabilidad	Instancias
Client	Arma los batches, los envía por TCP y recibe los resultados.	C
Gateway / MessageHandler	Acepta clientes, identifica cada sesión con un `client_id`, publica batches y EOF en las colas de entrada y devuelve los resultados.	G
Q1Filter	Filtra artículos hasta 2020 escritos en inglés (Q1).	N
Q2Filter	Filtra artículos hasta 2020 con sección "biography" / "biographie" (Q2).	N
ImageGetter	Descarga la imagen del artículo desde la API de Wikimedia.	N
Normalizer	Limpia el abstract (sin stopwords ni repetidas), cuenta palabras y separa los artículos hasta 2020 de los desde 2021.	N
WordReducer	Combina los conteos de las palabras de su partición: resuelve Q3 y aporta el top local de Q4.	P
TopMerger	Calcula el top 20 global a partir de los tops locales (Q4).	1
AverageAggregator	Calcula el promedio de palabras de abstract hasta 2020 inclusive.	1
Q5Filter	Deja los artículos desde 2021 con más palabras que el promedio (Q5).	N
Joiner	Reúne las respuestas de cada cliente esperando el EOF de cada consulta (P para Q3), las persiste y las devuelve.	1

Cuadro 1: Componentes del sistema. C clientes, G gateways, N réplicas que compiten por una cola y P particiones.

3 Escenarios

3.1 Casos de Uso

- El cliente envia el dataset y recibe los resultados de todas las consultas.
- Se ejecutan varios clientes a la vez.
- Al recibir un SIGTERM, cada proceso finaliza su trabajo, liberando recursos.



4 Vista Lógica

4.1 DAG

La lectura de artículos se divide en dos ramas, una donde están los filtros de las primeras dos consultas y otra donde se procesa el texto del resto.

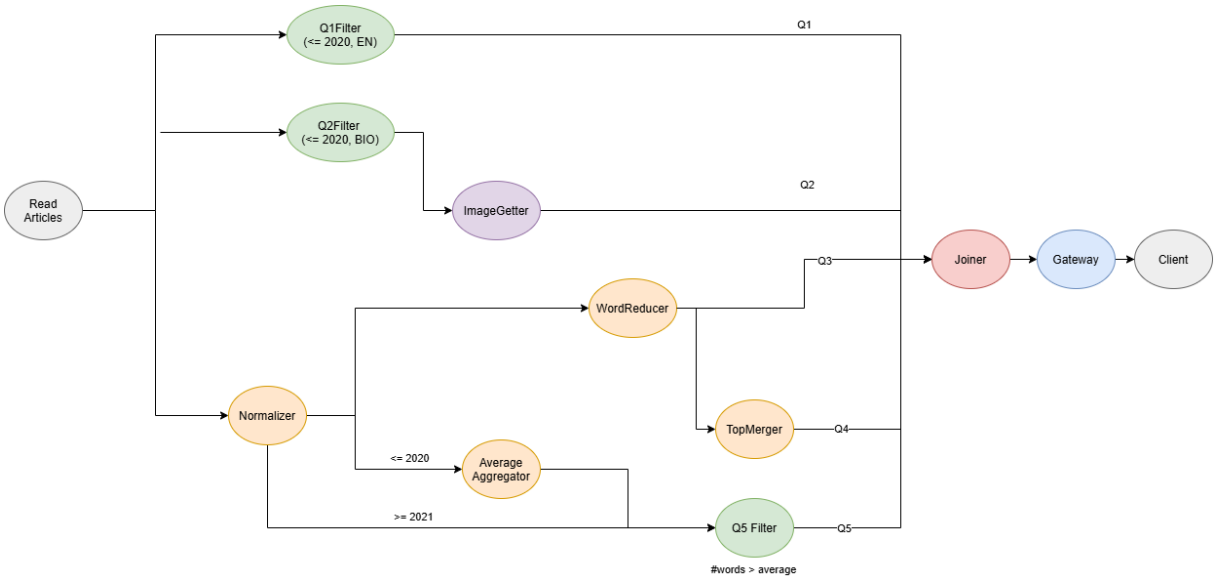


Figura 1: DAG

En la primera rama, `Q1Filter` deja los artículos en inglés hasta 2020 (Q1) y `Q2Filter` los que incluyen una sección de biografía, que pasan por `ImageGetter` para obtener la imagen (Q2). En la segunda rama, el `Normalizer` separa los artículos hasta 2020 de los posteriores: las palabras se reparten entre los `WordReducer`, que resuelven Q3 y alimentan al `TopMerger` (Q4); los artículos hasta 2020 alimentan al `AverageAggregator`, cuyo promedio se compara en `Q5Filter` con los artículos desde 2021 (Q5). Los cinco resultados convergen en el `Joiner`, que los entrega al Gateway y este al cliente.

5 Vista de Procesos

5.1 Diagrama de actividades

Muestra el flujo desde que el cliente arma los batches hasta que recibe los resultados. Se puede ver la división en las dos ramas y la unión en el Joiner. El Gateway reparte cada batch a las tres colas de entrada y, cuando el cliente termina, publica el EOF correspondiente a cada una. `Q5Filter` no procesa los artículos desde 2021 hasta que recibe el promedio. El Joiner persiste los resultados recién cuando recibió el EOF de cada consulta (en Q3, uno por cada partición de WordReducer).

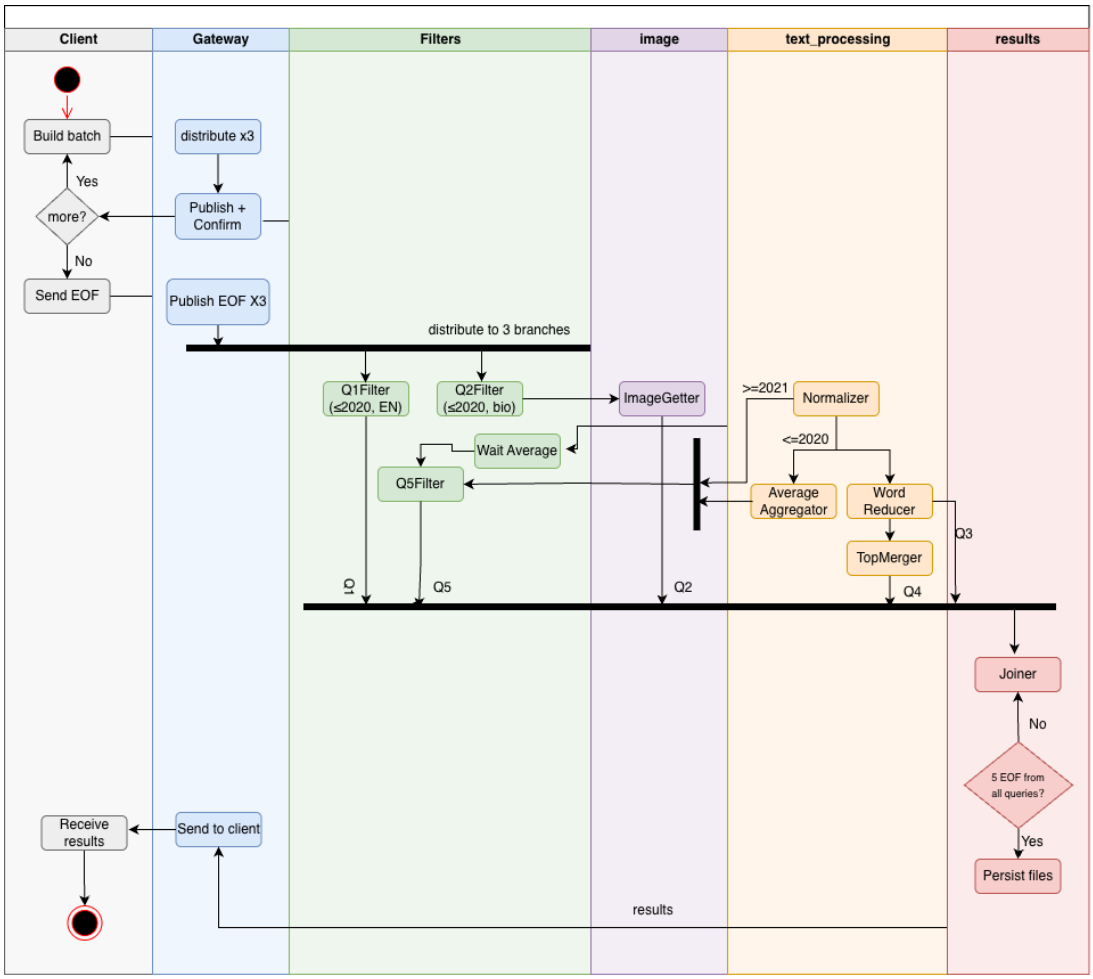


Figura 2: Diagrama de actividades

5.2 Diagrama de secuencia

El diagrama de secuencia muestra cómo se manejan los datos por etapa y cómo se llega a resolver cada consulta. Se distinguen tres momentos:

1. El procesamiento de cada batch.
2. La propagación del EOF (cada etapa espera el EOF de todas las réplicas anteriores antes de enviar el suyo).
3. El cálculo de los resultados que dependen de haber visto todos los datos: Q3 y Q4, por un lado, y el promedio con Q5 por el otro, que se ejecutan en paralelo.

El Joiner lleva la cuenta de EOF por consulta y por cliente: espera uno de Q1, Q2, Q4 y Q5, y P de Q3, uno por cada partición de WordReducer, identificadas por el campo `origin`. Los resultados de Q3 llegan en P partes que el Joiner une.

La figura agrupa las etapas relacionadas para facilitar la lectura; el diagrama completo, con todas las etapas por separado, se incluye en el Anexo.

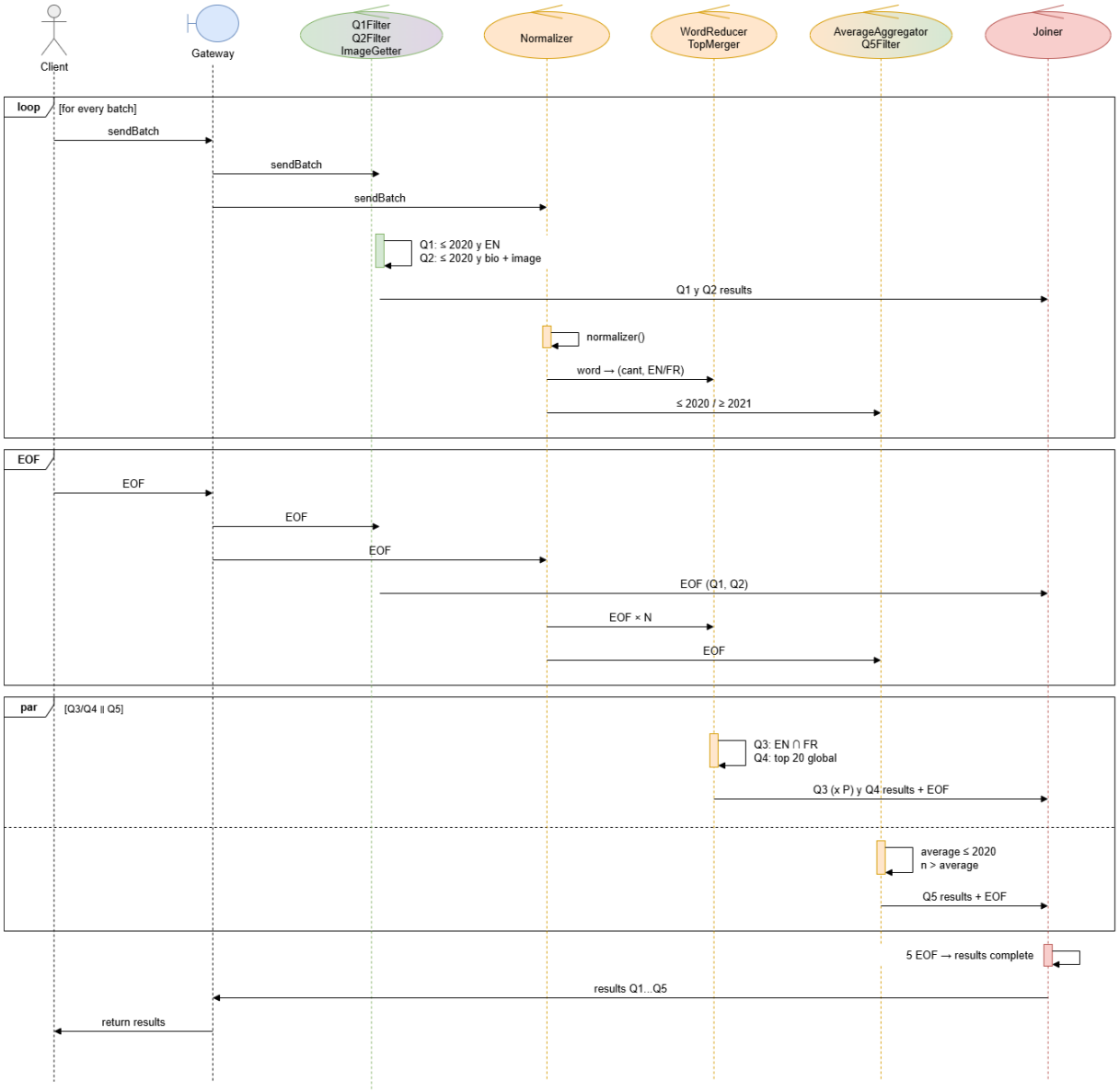


Figura 3: Diagrama de secuencia (versión corta).

6 Vista de Desarrollo

6.1 Diagrama de Paquetes

El código se organiza en paquetes por responsabilidad:

- `gateway` contiene el servidor TCP.
- `MessageHandler`; `filters` los filtros de Q1, Q2 y Q5.
- `text_processing` el Normalizer, el WordReducer, el TopMerger y el AverageAggregator.
- `image` el ImageGetter, que es el único que consume la API externa de Wikipedia por HTTPS.
- `results` el Joiner.

- `client` se comunica con el Gateway por TCP.

Todos los paquetes dependen de `common`, que agrupa lo compartido: `middleware` (productores y consumidores, colas de trabajo, exchanges y manejo de EOF), `protocol` (batches, artículos, resultados y serialización de mensajes) y `utils` (graceful shutdown ante SIGTERM, configuración, logging y el filtro por año). Los paquetes de negocio no dependen entre sí: `gateway` y `results`, por ejemplo, se comunican únicamente por las colas del middleware.

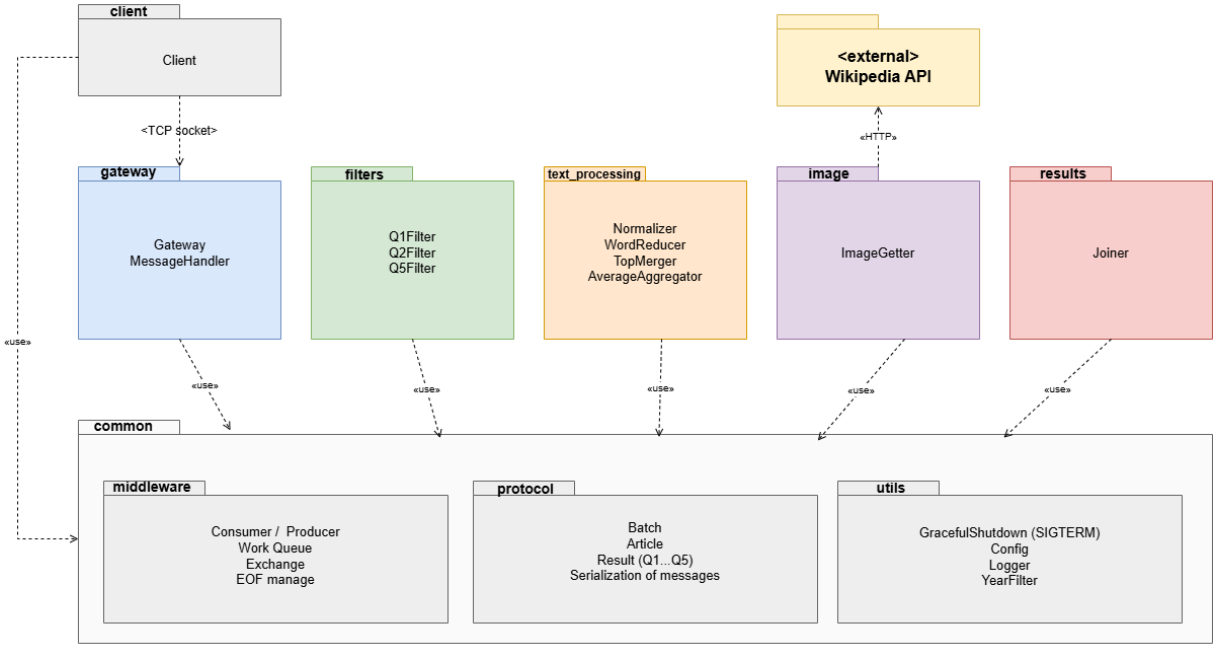


Figura 4: Diagrama de paquetes

7 Vista Física

7.1 Diagrama de Robustez

El diagrama muestra los procesos del sistema, las colas y exchanges que los conectan y los datos que intercambian.

- **Entrada:** El `MessageHandler` publica cada batch en `q1_in`, `q2_in` y `norm_in`, colas de trabajo de las que compiten las réplicas de cada etapa. Cada mensaje lleva un encabezado común con `client_id`, `batch_id`, origen e identificador del gateway.
- **Q1 y Q2:** `Q1Filter` y `Q2Filter` aplican el filtro por año y por idioma o sección. `Q2Filter` envía solo una referencia de imagen a `img_in`; `ImageGetter` descarga los bytes desde la API de Wikipedia.
- **Q3 y Q4:** El `Normalizer` limpia cada abstract (usa la lista de stopwords) y envía mapas parciales de palabras a `words_ex`, un exchange direct que particiona por `crc32(palabra) % P`, de modo que una misma palabra siempre llega al mismo `WordReducer`. Cada reducer resuelve Q3 para sus palabras y envía su top local a `top_in`; el `TopMerger` obtiene el top 20 global (Q4). Cada reducer envía al Joiner los resultados de Q3 de su partición y su propio EOF; el `TopMerger` espera además el EOF de las P particiones antes de calcular el top 20.

- **Q5:** El Normalizer manda a `avg_in` la suma y cantidad de palabras de los artículos hasta 2020, y a `q5_in` (una cola por cliente, declarada por el MessageHandler al abrir la sesión) los artículos desde 2021. El `AverageAggregator` publica el promedio en `avg_ex`, un exchange fanout que llega a todas las réplicas de `Q5Filter`, que recién entonces consumen `q5_in`.
- **Control:** Los exchanges `*_ctrl` permiten que las réplicas de una misma etapa se coordinen para el EOF antes de propagarlo.
- **Salida:** Todos los resultados llegan a `results_q`. El Joiner cuenta los EOF por consulta y por `client_id` (P para Q3, uno para el resto) y, cuando están completos, los persiste en un volumen compartido con el Gateway y publica en `out_q[g]` la ruta del resultado.

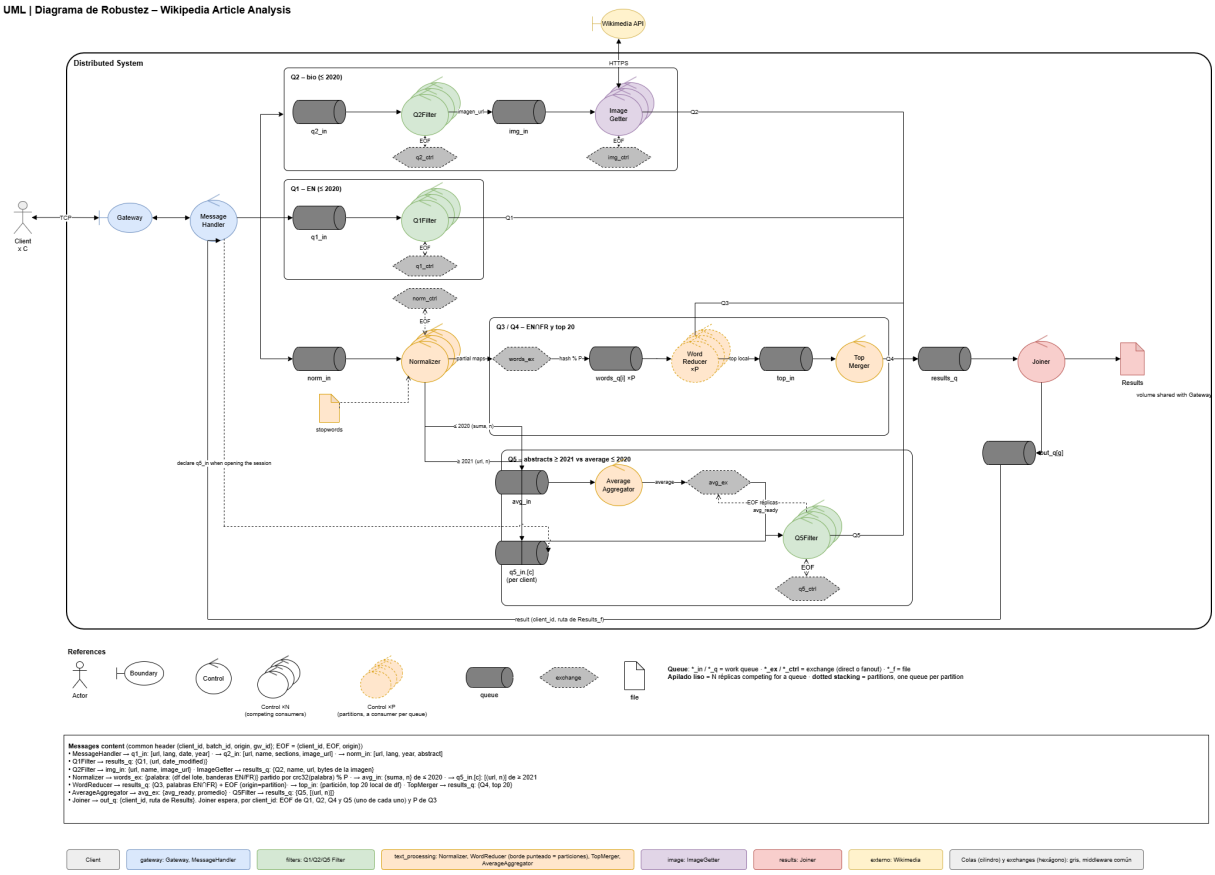


Figura 5: Diagrama de robustez

7.2 Diagrama de Despliegue

El sistema se despliega en nodos independientes, conectados a través del nodo de middleware (RabbitMQ). Los nodos de procesamiento no se comunican directamente entre sí, por lo que pueden replicarse sin modificar al resto.

Nodo	**Componentes**	**Réplicas**
Client PC	Client	varias
Gateway Node	Gateway, MessageHandler	varias
Middleware Node	RabbitMQ	1
Filter Node	Q1Filter, Q2Filter, Q5Filter	varias
Image Node	ImageGetter	varias
Normalizer Node	Normalizer	varias
Reducer Node	WordReducer	varias
Aggregator Node	TopMerger, AverageAggregator	1
Joiner Node	Joiner	1

Cuadro 2: Nodos del despliegue.

El cliente solo se conecta con el Gateway, y solo el `Image Node` accede a un servicio externo (API de Wikimedia).

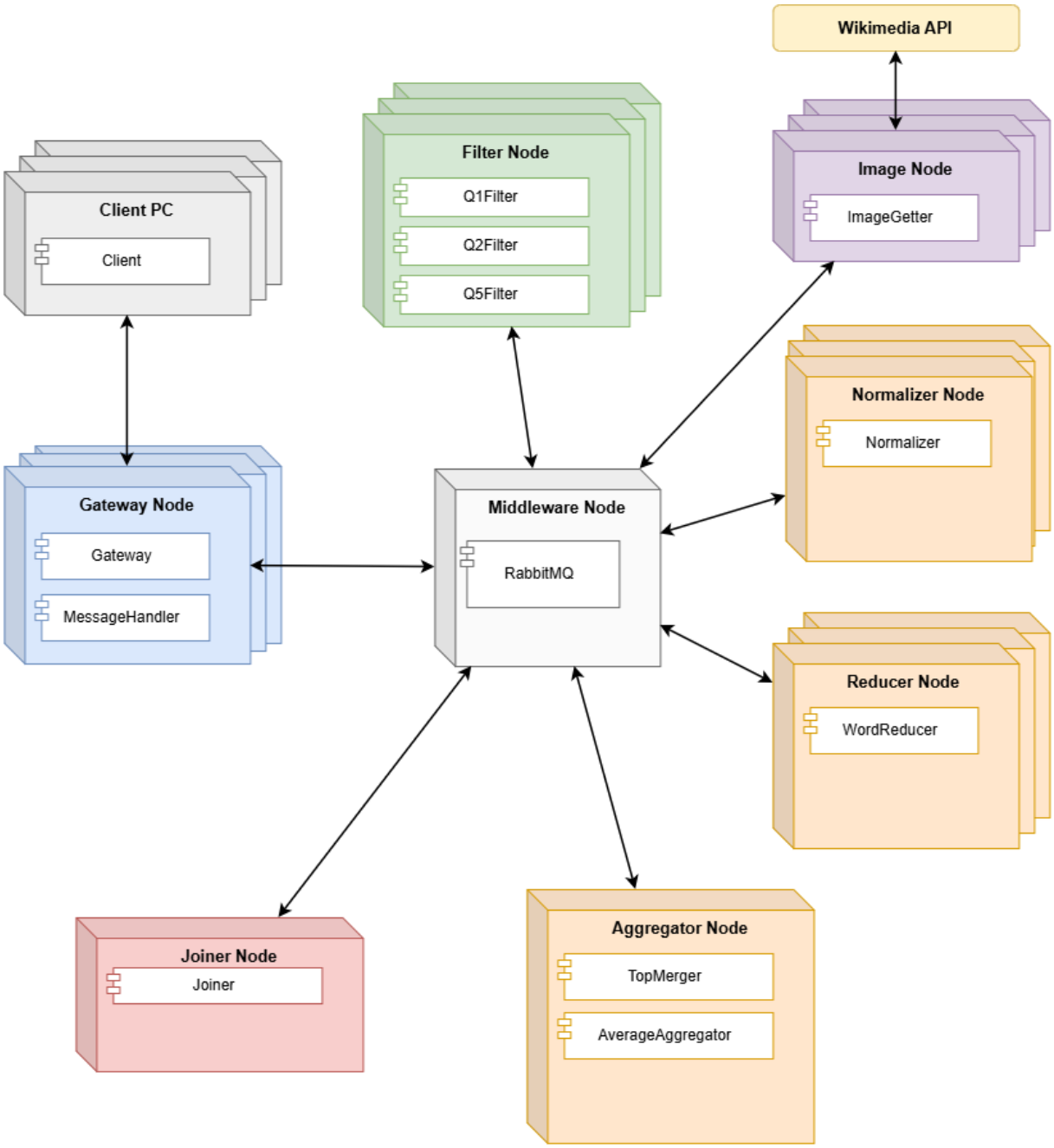
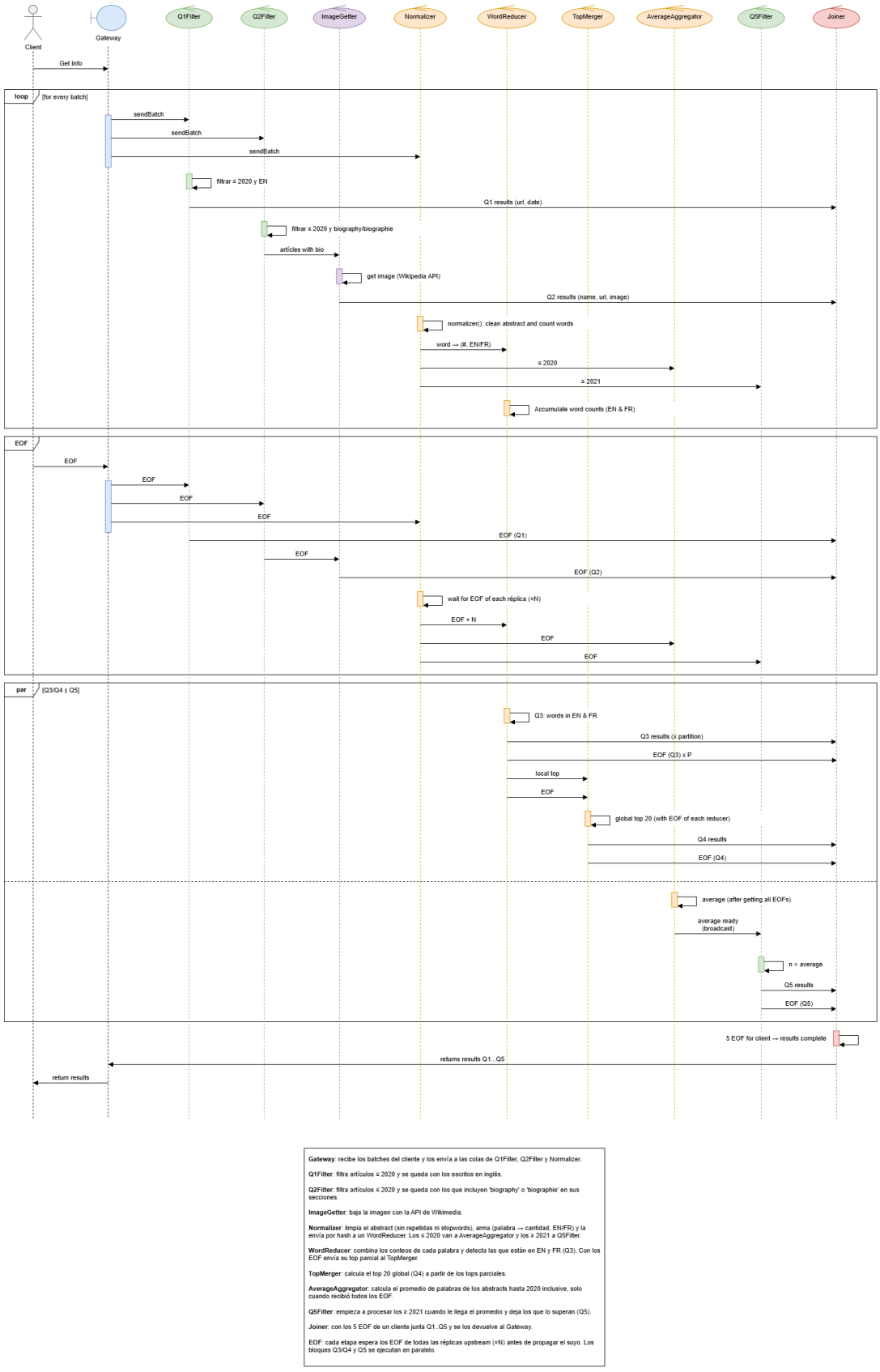


Figura 6: Diagrama de despliegue

8 Tareas y división entre integrantes

N.º	Tarea	Responsable
1	Middleware.	*Catalina Basso*
2	Protocolo.	*Catalina Basso*
3	Utils: configuración, logging y graceful shutdown (SIGTERM).	*Catalina Basso*
4	Client y Gateway/MessageHandler.	*Franco Favotti*
5	Q1Filter, Q2Filter e ImageGetter (Q1 y Q2).	*Victoria Fernandez Delgado*
6	Normalizer, WordReducer y TopMerger (Q3 y Q4).	*Victoria Fernandez Delgado*
7	AverageAggregator y Q5Filter (Q5).	*Franco Favotti*
8	Joiner y persistencia de resultados (conteo de EOF por consulta).	*Franco Favotti*
9	Despliegue.	*Victoria Fernandez Delgado*
10	Tests	Todos

A Diagrama de secuencia completo



Gateway: recibe los batches del cliente y los envía a las colas de Q1Filter, Q2Filter y Normalizer.

Q1Filter: filtra artículos ≤ 2020 y se queda con los escritos en inglés.

Q2Filter: filtra artículos ≤ 2020 y se queda con los que incluyen 'biography' o 'biographie' en sus secciones.

ImageGetter: baja la imagen con la API de Wikimedia.

Normalizer: limpia el abstract (sin repetidas ni stopwords), arma (palabra → cantidad, EN/FR) y la envía por hash a un WordReducer. Los ≤ 2020 van a AverageAggregator y los ≥ 2021 a Q5Filter.

WordReducer: combina los conteos de cada palabra y detecta las que están en EN y FR (Q3). Con los EOF envía su top parcial al TopMerger.

TopMerger: calcula el top 20 global (Q4) a partir de los tops parciales.

AverageAggregator: calcula el promedio de palabras de los abstracts hasta 2020 inclusive, solo cuando recibió todos los EOF.

Q5Filter: empieza a procesar los ≥ 2021 cuando le llega el promedio y deja los que lo superan (Q5).

Joiner: con los 5 EOF de un cliente junta Q1..Q5 y se los devuelve al Gateway.

EOF: cada etapa espera los EOF de todas las réplicas upstream (×N) antes de propagar el suyo. Los bloques Q3/Q4 y Q5 se ejecutan en paralelo.

Figura 7: Diagrama de secuencia completo, con todas las etapas por separado.